

Kalender-App für Android

Verwaltung der Termine des Standardkalenders

Maximilian Ekgardt · Szymon Moleda · Tim Seifried

DHBW · 19.08.2026

Vue.js

Ionic

TypeScript

Capacitor

Zielsetzung

Lauffähigkeit

Läuft auf einem echten Android-Gerät

Berechtigungen

Kalenderzugriff wird zur Laufzeit angefordert

Termine anzeigen

Termine des Standardkalenders werden gelistet

Termine verwalten

Neue Termine erstellen, bestehende löschen

Orte weiterverwenden

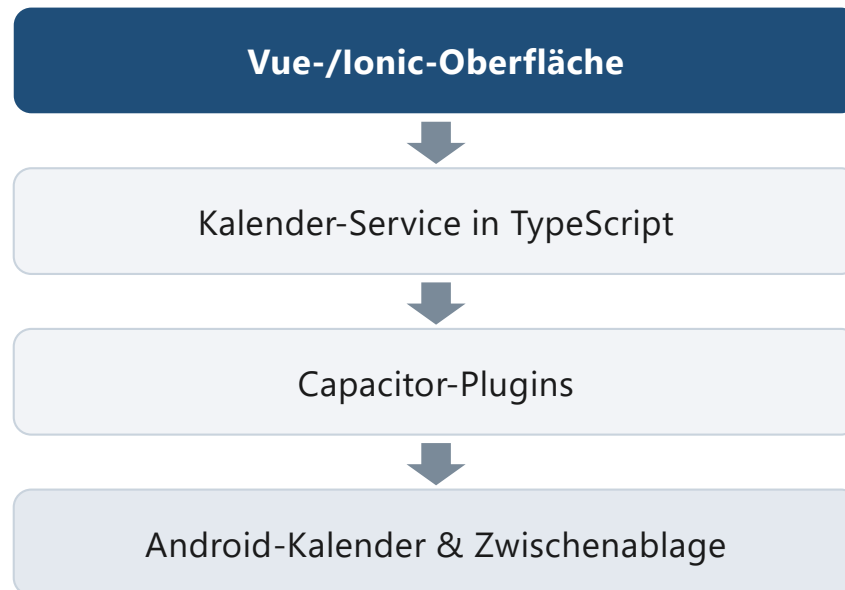
Ort in Google Maps öffnen oder kopieren

Reines Vue.js

Oberfläche vollständig aus Vue-Komponenten

Technischer Aufbau

Schichten



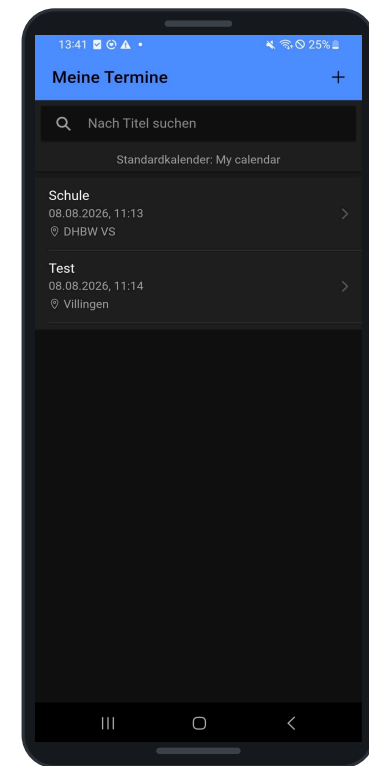
Drei Hauptansichten



Terminliste

Funktionen der Listenansicht

- Termine des erkannten Standardkalenders
- Sortierung nach Startdatum, aufsteigend
- Freitextsuche nach dem Titel (Zusatzanforderung)
- Anzeige von Datum und Ort
- Navigation zur Detailansicht, Schaltfläche „Hinzufügen“



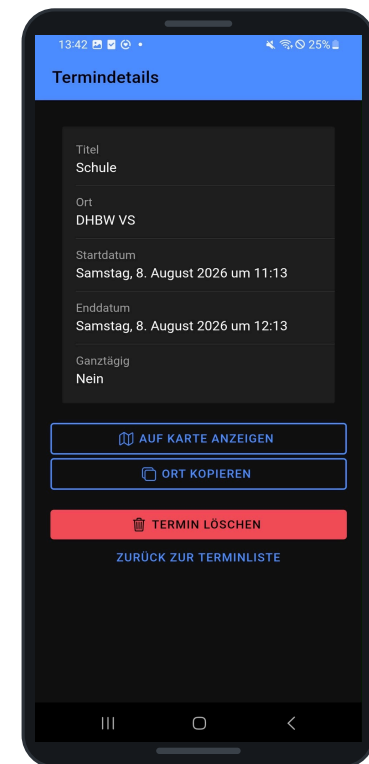
Detailansicht

Angezeigte Informationen

- Titel, Ort, Start- und Enddatum
- Ganztägig: Ja oder Nein

Weitere Aktionen

- Ort in Google Maps öffnen
- Ort in die Zwischenablage kopieren
- Termin löschen – mit eigenem Bestätigungsdialog
- Dialog als Vue-Komponente, kein natives Modal



Neue Termine erstellen

Eingabefelder

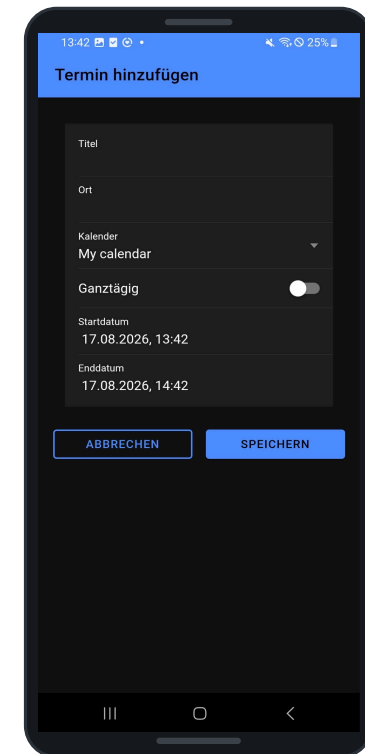
- Titel und Ort
- Zielkalender (Zusatzanforderung)
- Startdatum, Enddatum, Ganztägig

Aktionen

- Abbrechen und Speichern
- Prüfung der Eingaben vor dem Speichern

Validierung

- Titel darf nicht leer sein
- Kalender muss ausgewählt sein
- Enddatum muss nach dem Startdatum liegen



Verwendete native Schnittstellen

Calendar-Plugin

[@ebarooni/capacitor-calendar](#)

Verwendete Funktionen

- Kalenderberechtigung anfordern
- Standardkalender ermitteln
- Verfügbare Kalender auflisten
- Termine eines Zeitraums laden
- Termin erstellen und löschen

Clipboard-Plugin

[@capacitor/clipboard](#)

Verwendete Funktion

- Ort eines Termins in die Zwischenablage schreiben

Beim Erstellen wird ein Datenobjekt mit Titel, Ort, Kalender-ID, Start, Ende und Ganztägig-Status an das Plugin übergeben.

Android-Berechtigungen

AndroidManifest.xml

```
<uses-permission android:name=
    "android.permission.READ_CALENDAR" />
<uses-permission android:name=
    "android.permission.WRITE_CALENDAR" />
```

Wozu?

- READ_CALENDAR: Termine anzeigen
- WRITE_CALENDAR: erstellen und löschen

Manifest-Einträge allein reichen nicht aus

Kalenderdaten sind persönliche Informationen: die Berechtigungen müssen zusätzlich zur Laufzeit angefordert werden. Bei Verweigerung zeigt die App eine verständliche Fehlermeldung.

Problem 1: Datumsabfrage

Problem

Das Calendar-Plugin meldete:

`From date must be provided.`

... obwohl ein Startdatum übergeben wurde.

Ursache

- Der Wert 0 wurde über die Android-Bridge nicht als Java-Long erkannt
- Das Plugin behandelte ihn wie einen fehlenden Parameter

Lösung

- Vollständiger Millisekunden-Zeitstempel
- Abfrage beginnt am 01.01.2000
- Prüfung über ADB und Android-Geräteprotokoll

Problem 2: Standardkalender

Problem

Jedes Gerät hat einen anderen Standardkalender.

- Gerät 1: lokaler Kalender „My calendar“
- Gerät 2: Google-Kalender

Ursache

Der Standardkalender wird durch das Gerät beziehungsweise das Betriebssystem bestimmt – nicht durch die App.

Lösung

Keine fest hinterlegte Kalender-ID:

`getDefaultCalendar()`

- Dynamische Abfrage bei jedem Laden
- Filterung anhand der zurückgegebenen Kalender-ID
- Test auf zwei Android-Geräten

Plattformen und Einschränkungen

Android

getestet

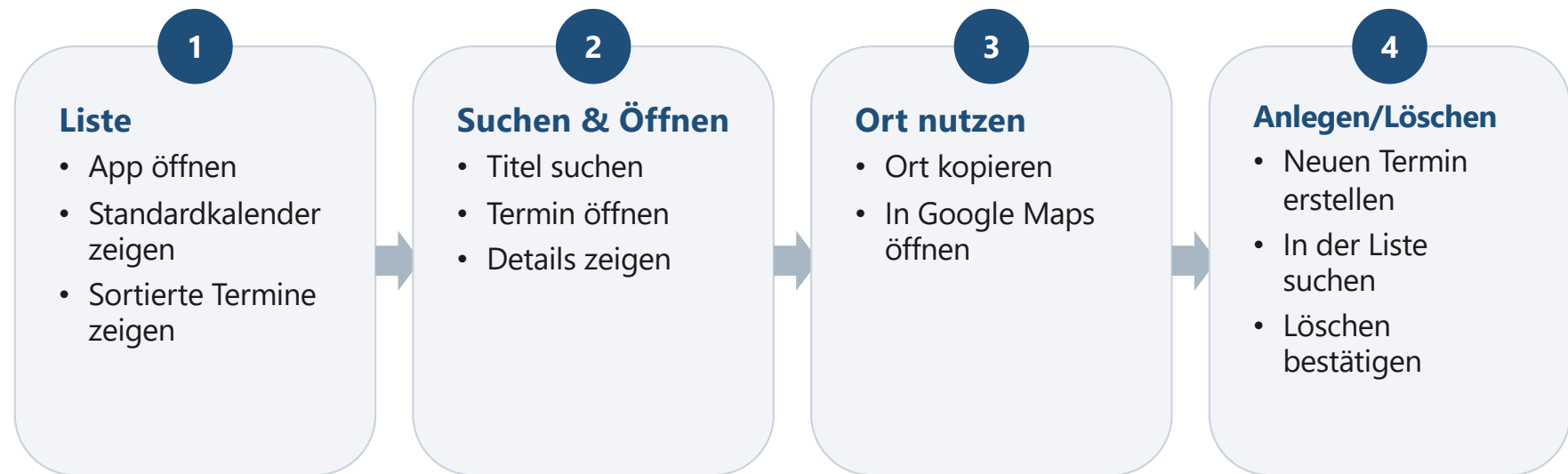
- Auf zwei echten Geräten getestet
- Kein Kalenderzugriff in der Browser-Vorschau
- Echtes Gerät oder eingerichteter Emulator nötig
- Kalender müssen auf dem Gerät synchronisiert sein
- Google Maps wird extern geöffnet

iOS

offen

- Vom Calendar-Plugin grundsätzlich unterstützt
- In diesem Projekt nicht eingerichtet und nicht getestet
- Zusätzliche iOS-Konfiguration erforderlich
- Berechtigungsbeschreibung in der Info.plist nötig

Live-Demo



Was war außerdem zu beachten?

Technische Fallstricke

- Datum und Uhrzeit als Millisekunden-Zeitstempel
- Termine nur über ihre ID ansprechbar
- Ganztagstermine brauchen eigene Darstellung
- Enddatum muss nach dem Startdatum liegen

Geräte und Rechte

- Native Funktionen nur auf echten Geräten prüfbar
- Nutzer können Berechtigungen verweigern
- Kalender-IDs unterscheiden sich je Gerät
- Mehrere lokale, Google- oder abonnierte Kalender möglich

Qualitätssicherung

Automatisierte Prüfungen

- ✓ Produktions-Build erfolgreich
- ✓ TypeScript-Prüfung ohne Fehler
- ✓ ESLint ohne Fehler
- ✓ Unit-Test erfolgreich
- ✓ Android-Debug-Build erfolgreich

Praxistests

- ✓ Test auf zwei Android-Geräten
- ✓ Test mit lokalem Standardkalender
- ✓ Test mit Google-Standardkalender

Fazit

Ergebnis

- ✓ Alle Grundanforderungen umgesetzt
- ✓ Beide Zusatzanforderungen umgesetzt
- ✓ Native Kalenderverwaltung funktioniert
- ✓ Auf mehreren Android-Geräten getestet
- ✓ Einfache und übersichtliche Bedienung

Mögliche Erweiterungen

- Bearbeiten vorhandener Termine
- Kalenderfilter und Zeitraumauswahl in der Liste
- Erinnerungen und Beschreibungen
- Unterstützung und Tests für iOS
- Veröffentlichung als signierte Release-App

Vielen Dank für die Aufmerksamkeit.